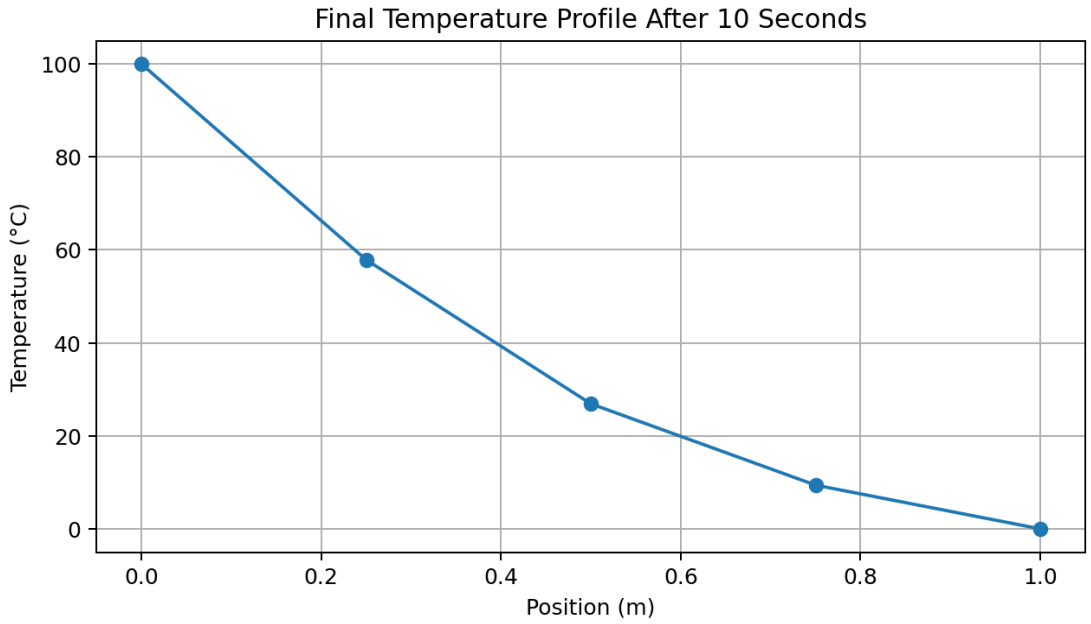
Numerical Simulation of Heat Transfer in a One-Dimensional Rod Using Python

A beginner-friendly numerical methods project

Prepared by: Muhammad Awais

Institution: Government College University Faisalabad



Project report • 16 October 2026

1. Abstract

This project models heat transfer along a one-dimensional rod using a finite-difference method. A short Python program updates the temperature at interior grid points over time and plots the final temperature profile. The project connects a partial differential equation with a simple numerical algorithm and visualization.

2. Introduction

Heat transfers from warmer regions to cooler regions. The heat equation is a mathematical model used to describe how temperature changes with position and time. Numerical methods are useful when a model is solved approximately on a discrete grid.

3. Objectives

- Understand the basic one-dimensional heat equation.
- Implement a finite-difference update in Python.
- Keep the boundary temperatures fixed while updating interior points.
- Visualize the calculated temperature distribution.

4. Mathematical Model

The one-dimensional heat equation is:

$$\partial T/\partial t = \alpha \, \partial^2 T/\partial x^2$$

Here, T is temperature, t is time, x is position, and α is thermal diffusivity. The explicit finite-difference formula used in the code is:

$$T_i^{n+1} = T_i^n + r\,(T_{i+1}^n - 2T_i^n + T_{i-1}^n)$$

where $r = \alpha \Delta t/(\Delta x)^2$. For this example, $\alpha = 0.01$ m²/s, $\Delta t = 0.5$ s, and $\Delta x = 0.25$ m, giving $r = 0.08$. This satisfies the usual one-dimensional explicit stability condition $r \leq 0.5$.

5. Problem Setup

Parameter	Value
Rod length	1.0 m
Grid positions	0, 0.25, 0.50, 0.75, 1.00 m
Initial temperature	100°C at left endpoint; 0°C elsewhere
Boundary conditions	Left end fixed at 100°C; right end fixed at 0°C
Time step	0.5 s
Number of steps	20 (10 seconds total)
Thermal diffusivity	0.01 m²/s (illustrative value)

6. Python Implementation

Save the following code as **heat_transfer.py**. It requires Python 3 and Matplotlib. The two endpoint values remain fixed because the loop updates only indices 1, 2, and 3.

```
import matplotlib.pyplot as plt

# Model settings
x = [0, 0.25, 0.50, 0.75, 1.00]  # position in metres
T = [100, 0, 0, 0, 0]            # initial temperature in Celsius
```

```
alpha = 0.01  # illustrative thermal diffusivity (m^2/s)
dx = 0.25
dt = 0.5
r = alpha * dt / dx**2

# Calculate temperature for 20 time steps
for step in range(20):
    new_T = T.copy()

    # Update only the three interior points; ends stay fixed
    for i in range(1, 4):
        new_T[i] = T[i] + r * (T[i+1] - 2*T[i] + T[i-1])

    T = new_T

print("Final temperature (C):")
print([round(value, 2) for value in T])

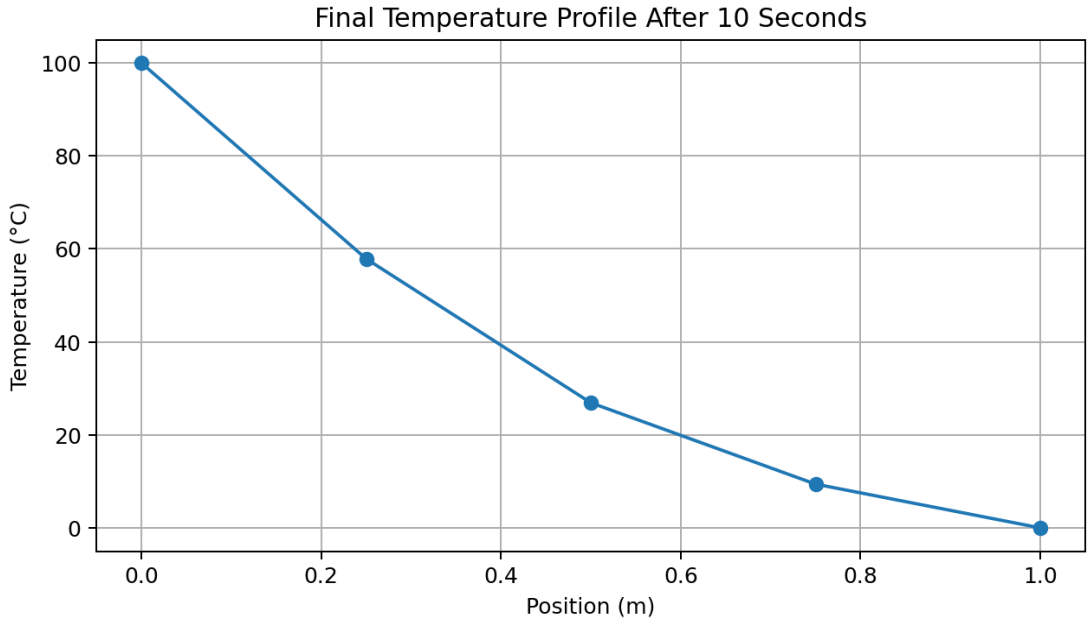
# Plot the final temperature profile
plt.plot(x, T, marker="o")
plt.xlabel("Position (m)")
plt.ylabel("Temperature (C)")
plt.title("Heat Transfer in a One-Dimensional Rod")
plt.grid(True)
plt.tight_layout()
plt.show()
```

7. Results

After 20 time steps (10 seconds), the calculated temperature values, rounded to two decimal places, are:

Position (m): 0.00, 0.25, 0.50, 0.75, 1.00

Temperature (°C): 100.00, 57.88, 26.91, 9.41, 0.00



The left endpoint remains at 100°C and the right endpoint remains at 0°C. Interior temperatures change as the simulation proceeds. The curve represents this simplified numerical model, not a measured experiment.

8. Discussion

The code demonstrates how a differential equation can be approximated using a numerical update. Each new interior temperature is calculated from the previous temperatures at that point and its two neighbours. A copy of the temperature list is used so that all updates for one time step use the previous time-step values.

9. Limitations

The grid contains only five points, so the simulation is coarse and results are approximate. The thermal diffusivity is an illustrative value and is not assigned to a specific material. The model assumes fixed endpoint temperatures and omits sources of heat, convection, and other physical effects. A more advanced project could test finer grids and compare with an analytical solution.

10. Conclusion

This project gives a compact example of applied mathematics and Python. It applies a finite-difference method to a one-dimensional heat equation, calculates temperature changes, and displays the result graphically.

References

1. Incropera, F. P., DeWitt, D. P., Bergman, T. L., and Lavine, A. S. *Fundamentals of Heat and Mass Transfer*. Wiley.
2. Chapra, S. C., and Canale, R. P. *Numerical Methods for Engineers*. McGraw-Hill.
3. Python documentation: https://docs.python.org/3/
4. Matplotlib documentation: https://matplotlib.org/

Reproducibility note: The Python source file, README, requirements, citation metadata, license, and graph are included in the companion GitHub/Zenodo-ready project archive.